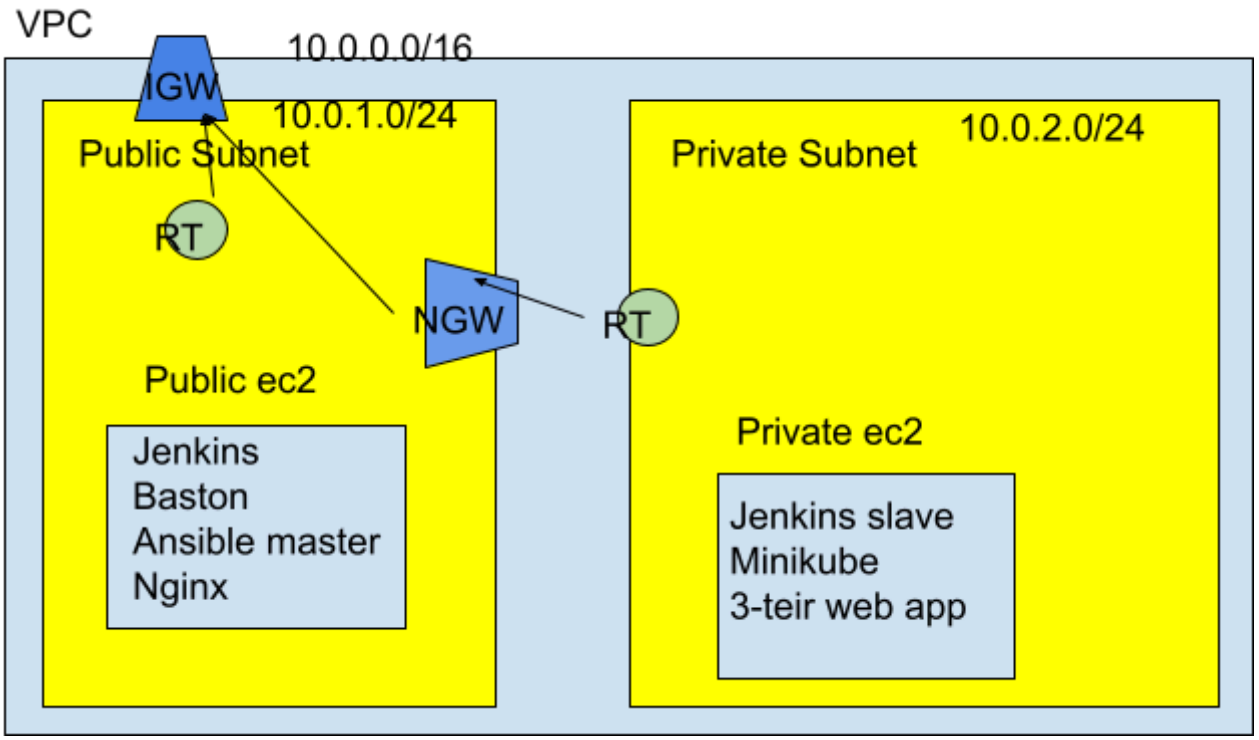
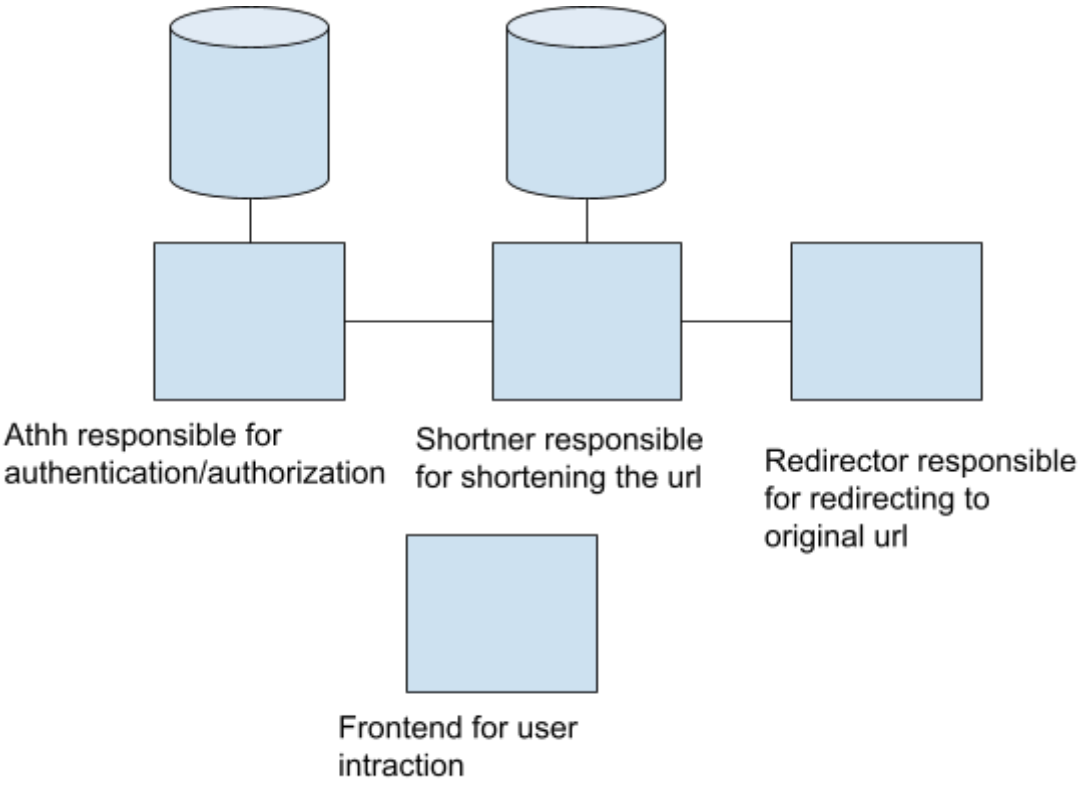
AWS Infrastructure



- The security groups are omitted in this diagram
- Terraform is used to manage the infra code can be found here

3 Tier application

- A url shortener application was used called link shrink code here
- This application is architected into microservice architecture with 3 backend micro service 1 frontend microservice and 2 database instances
- The HLD of app is shown below



Dockerizing the application

- Docker files for all components were written
- A bash script was written to automate the image creation process
- Some samples of docker file are shown here full code can be found here

Dockerfile

authn > Dockerfile

```
FROM amazoncorretto:17

WORKDIR /app

COPY build/libs/authn.jar .

ENV SPRING_PROFILES_ACTIVE=DBh2
CMD ["echo","$SPRING_DATASOURCE_URL"]
ENTRYPOINT ["java", "-jar", "/app/authn.jar"]

```

Dockerfile

Docker > database > authn > Dockerfile

```
FROM docker.io/library/postgres:latest

# Set environment variables for PostgreSQL
ENV POSTGRES_USER=backend
ENV POSTGRES_PASSWORD=pass_876
ENV POSTGRES_DB=linkshrink

# Copy SQL scripts to the initialization directory
COPY Docker/database/authn/sql/ /docker-entrypoint-initdb.d/

# Expose the default PostgreSQL port
EXPOSE 5432
```

buildImages.sh

build-scripts > buildImages.sh

```
set -eux
docker build -t alquama00s/linkshrink/authndb:$1 -t alquama00s/linkshrink/authndb:latest -f Docker/database/authn/Doc
docker build -t alquama00s/linkshrink/redirectordb:$1 -t alquama00s/linkshrink/redirectordb:latest -f Docker/database
docker build -t alquama00s/linkshrink/frontend:$1 -t alquama00s/linkshrink/frontend:latest -f Docker/frontend/Dockerf
docker build -t alquama00s/linkshrink/shortner:$1 -t alquama00s/linkshrink/shortner:latest -f shortner/Dockerfile shor
docker build -t alquama00s/linkshrink/redirector:$1 -t alquama00s/linkshrink/redirector:latest -f redirector/Dockerfi
docker build -t alquama00s/linkshrink/authn:$1 -t alquama00s/linkshrink/authn:latest -f authn/Dockerfile authn

```

Kubernetes object files for k8s deployment

- Kubernetes object yml files were written for all services and databases
- A bash script was written to automate the k8s deployment process
- Some samples of k8s yml file are shown here full code can be found here

authn.yml

k8s > backend > authn.yml > {} spec > {} template > {} spec

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: authn-deployment
  labels:
    app: authn
spec:
  replicas: 1
  selector:
    matchLabels:
      app: authn
  template:
    metadata:
      labels:
        app: authn
    spec:
      containers:
      - name: authn
        image: alquama00s/linkshrink/authn:TAG
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 8080
        env:
        - name: SERVER_PORT
          value: "8080"
        - name: SPRING_DATASOURCE_USERNAME
```

app.sh

k8s > app.sh

```
set -eux
kubectl apply -f database/
kubectl apply -f backend/
kubectl apply -f frontend/
```

Jenkins file

- A Jenkins file was written to automate build image creation and deployement

Jenkinsfile

build-scripts > Jenkinsfile

```
pipeline{
    agent {
        label 'server'
    }
    tools{
        gradle "gradle-7"
    }
    environment{
        JAVA_HOME = '/usr/lib/jvm/java-17-openjdk-amd64'
    }
      stages{
          stage ('Checkout'){
              steps{
                  cleanWs()
                  sh "echo $JAVA_HOME"
                  sh "git clone https://github.com/alquama00s/link_shrir
                  sh "ls -la"
              }
          }
          stage('Build'){
              steps{
                  nodejs(nodeJSInstallationName: 'NodeJS24') {
                      sh 'bash build-scripts/build.sh'
                  }
                  sh "docker rmi -f \$(docker images -q) || echo 'some p
                  sh "bash build-scripts/buildImages.sh 3.0.$BUILD_NUMBE
              }
          }
          stage('Process Images'){
                      steps{
                          sh "docker image ls"
                          sh "bash build-scripts/loadImages.sh 3.0.$BUII
                      }
                  }
          stage('Deploy'){
              steps{
                  echo 'Deploying....'
                  sh "find k8s -type f -name '*.yml' -exec sed -i.bak 's
                  sh "cd k8s && bash app.sh"
                  archiveArtifacts artifacts: 'k8s/**/*.yml', allowEmpty
```

Creating infrastructure using terraform

- terraform files were written for automating infra creation
- Some samples of tf file are shown here full code can be found here

```hcl
resource "aws_key_pair" "my_key" {
  key_name   = "my-key-pair"
  public_key = file("public_ec2.pub")
}

resource "aws_instance" "public_instance" {
  ami           = "ami-0360c520857e3138f" # Example AMI, update as needed
  instance_type = "t2.micro"
  subnet_id     = aws_subnet.public.id
  security_groups = [aws_security_group.public_sg.id]
  key_name      = aws_key_pair.my_key.key_name
}

resource "aws_instance" "private_instance" {
  ami           = "ami-0360c520857e3138f" # Example AMI, update as needed
  instance_type = "t2.micro"
  subnet_id     = aws_subnet.private.id
  security_groups = [aws_security_group.private_sg.id]
  key_name      = aws_key_pair.my_key.key_name
}

output "public_instance_ip" {
  value = aws_instance.public_instance.publi
}
```

Null annotation types have been detected in the project.
Do you wish to enable null analysis for this project?

- Terraform plan & apply was run

```
      + enable_dns_hostnames                = (known after apply)
      + enable_dns_support                  = true
      + enable_network_address_usage_metrics = (known after apply)
      + id                                  = (known after apply)
      + instance_tenancy                    = "default"
      + ipv6_association_id                 = (known after apply)
      + ipv6_cidr_block                     = (known after apply)
      + ipv6_cidr_block_network_border_group = (known after apply)
      + main_route_table_id                 = (known after apply)
      + owner_id                            = (known after apply)
      + region                              = "us-east-1"
      + tags_all                            = (known after apply)
    }

Plan: 15 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + private_instance_id = (known after apply)
  + public_instance_ip  = (known after apply)
```

```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

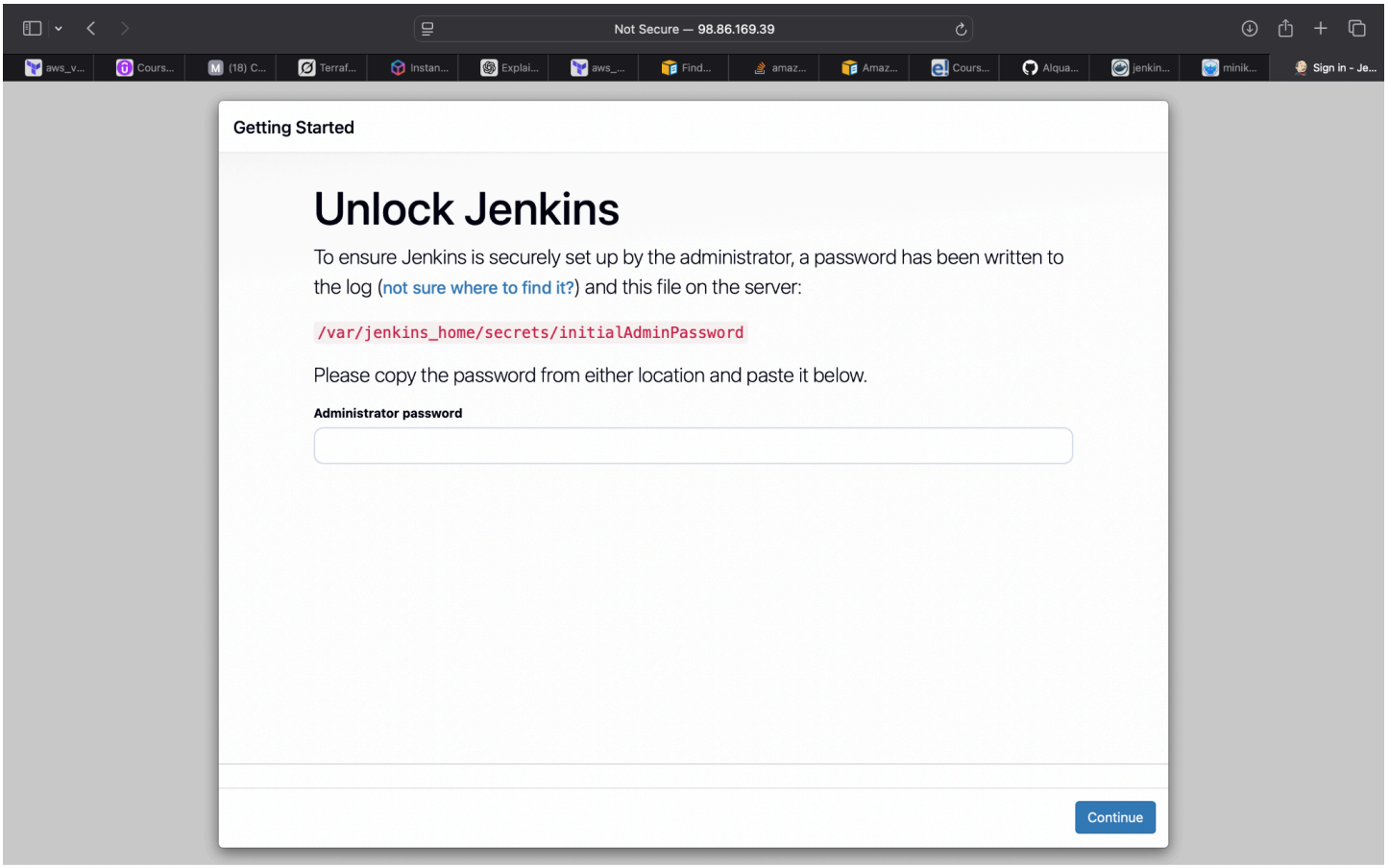
aws_instance.public_instance: Creating...
aws_instance.private_instance: Creating...
aws_instance.public_instance: Still creating... [00m10s elapsed]
aws_instance.private_instance: Still creating... [00m10s elapsed]
aws_instance.public_instance: Still creating... [00m20s elapsed]
aws_instance.private_instance: Still creating... [00m20s elapsed]
aws_instance.private_instance: Still creating... [00m30s elapsed]
aws_instance.public_instance: Still creating... [00m30s elapsed]
aws_instance.public_instance: Creation complete after 38s [id=i-029c4a0d7a056c07d]
aws_instance.private_instance: Creation complete after 38s [id=i-07fdd42c476f1acee]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

private_instance_id = "i-07fdd42c476f1acee"
public_instance_ip = "98.86.182.121"
alquamasalim@Alquamas-MacBook-Air iac-aws %
```

- Jenkins was installed on docker & pipeline was configured



- Minikube was installed on private instance and app was deployed

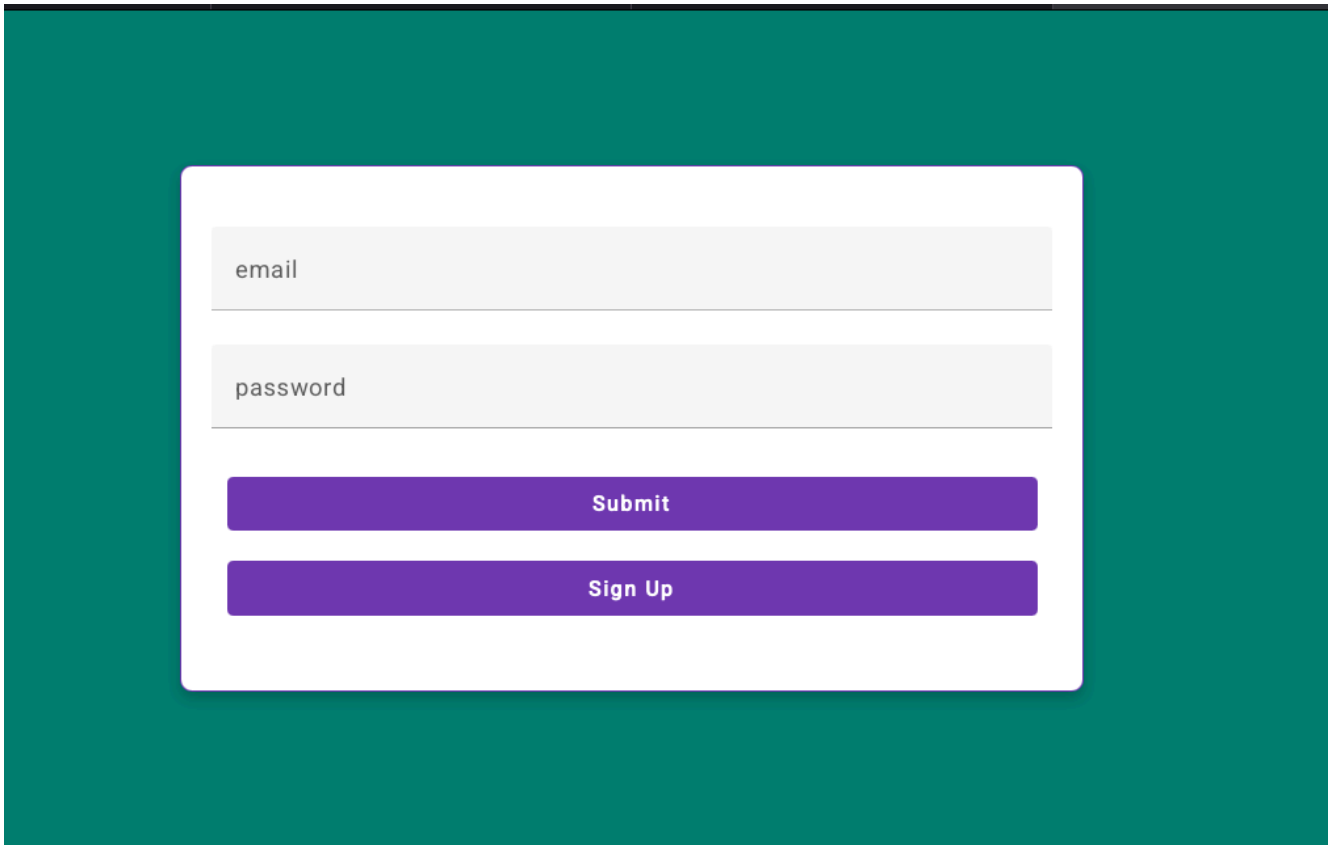
```
ubuntu@ip-10-0-2-188:~$ minikube
minikube provisions and manages local Kubernetes clusters optimized for development workflows.

Basic Commands:
  start            Starts a local Kubernetes cluster
  status           Gets the status of a local Kubernetes cluster
  stop             Stops a running local Kubernetes cluster
  delete           Deletes a local Kubernetes cluster
  dashboard        Access the Kubernetes dashboard running within the minikube cluster
  pause            pause Kubernetes
  unpause          unpause Kubernetes

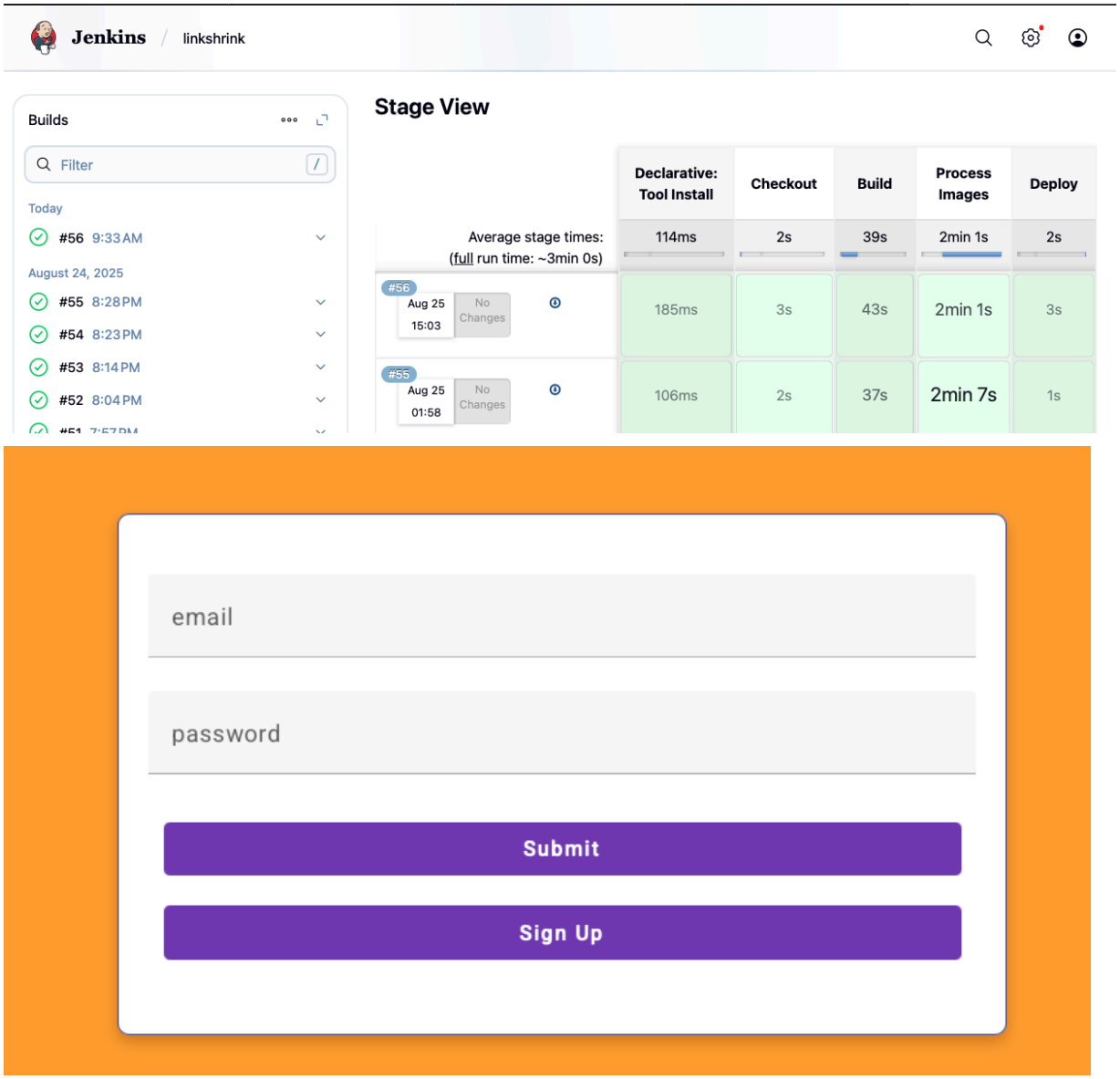
Images Commands:
  docker-env       Provides instructions to point your terminal's docker-cli to the Docker Engine inside minikube.
(Useful for building docker images directly inside minikube)
  podman-env       Configure environment to use minikube's Podman service
  cache            Manage cache for images
  image            Manage images

Configuration and Management Commands:
  addons           Enable or disable a minikube addon
  config           Modify persistent configuration values
  profile          Get or list the current profiles (clusters)
  update-context   Update kubeconfig in case of an IP or port change
```

- Application was accessed using nginx and checked



- The color of the ui was changed and it was deployed automatically and verified



Clean Up

- terraform destroy was run to delete the aws infra
- Following are the output

```
aws_key_pair.my_key: Destroying... [id=my-key-pair]
aws_security_group.public_sg: Destroying... [id=sg-03db3a5cb5d5133b3]
aws_nat_gateway.natgw: Still destroying... [id=nat-0acf37f07624a9731, 00m30s elapsed]
aws_internet_gateway.igw: Still destroying... [id=igw-0c09c8ae9a366c495, 00m30s elapsed]
aws_key_pair.my_key: Destruction complete after 2s
aws_subnet.private: Destruction complete after 2s
aws_security_group.private_sg: Destruction complete after 2s
aws_security_group.public_sg: Destruction complete after 2s
aws_internet_gateway.igw: Still destroying... [id=igw-0c09c8ae9a366c495, 00m40s elapsed]
aws_nat_gateway.natgw: Still destroying... [id=nat-0acf37f07624a9731, 00m40s elapsed]
aws_internet_gateway.igw: Still destroying... [id=igw-0c09c8ae9a366c495, 00m50s elapsed]
aws_nat_gateway.natgw: Still destroying... [id=nat-0acf37f07624a9731, 00m50s elapsed]
aws_nat_gateway.natgw: Still destroying... [id=nat-0acf37f07624a9731, 01m00s elapsed]
aws_internet_gateway.igw: Still destroying... [id=igw-0c09c8ae9a366c495, 01m00s elapsed]
aws_nat_gateway.natgw: Destruction complete after 1m3s
aws_eip.nat: Destroying... [id=eipalloc-060885416f3d6f5f8]
aws_subnet.public: Destroying... [id=subnet-0f0fa1e7b934c935d]
aws_internet_gateway.igw: Destruction complete after 1m4s
aws_subnet.public: Destruction complete after 1s
aws_vpc.main: Destroying... [id=vpc-070abf2c48e1908fc]
aws_eip.nat: Destruction complete after 2s
aws_vpc.main: Destruction complete after 3s

Destroy complete! Resources: 15 destroyed.
alquamasalim@Alquamas-MacBook-Air iac-aws %
```